

自适应记忆在编程任务中的优化

最终实验报告 | 题目三 C | 2026-09-11

最终结论：方案已经形成可运行、可回放、可回退的编程记忆优化闭环；历史长对话回放出现质量与成本正收益，但冻结切分未守住 evidence recall，因此候选策略不晋升，Top-10 继续作为默认安全策略。编程侧尚无稳定 SWE-bench pass@1 提升证据。

923 历史回放 query	296 冻结验证 query	17 + 14 TS + Python 测试	1 / 5 SWE resolved
-------------------	-------------------	---------------------------	-----------------------

交付判断

问题	验证	交付决策
记忆能否在长对话中带来收益	历史回放为正；冻结验证出现 recall 下降	保留候选，不默认启用
编程反馈能否驱动策略	compile、tests、regression、pass@1 和 token 已进入配对反馈	机制与单测完成
是否已证明提高编程成功率	5 题 SWE 对照未增加 resolved	不作成功率提升主张
能否安全交付	默认关闭、shadow 学习、硬失败降级、干净包复现通过	可提交

代码基线：TencentDB Agent Memory v2.0.0-beta.1。交付分支：topic3c-adaptive-memory。

1. 研究问题与范围

固定 Top-10 召回稳定但成本较高，固定缩小 K 又可能丢失关键证据。编程任务还要求补丁可应用、可编译、通过目标测试且不引入回归，仅用文本相似度或答案 F1 作为反馈并不充分。

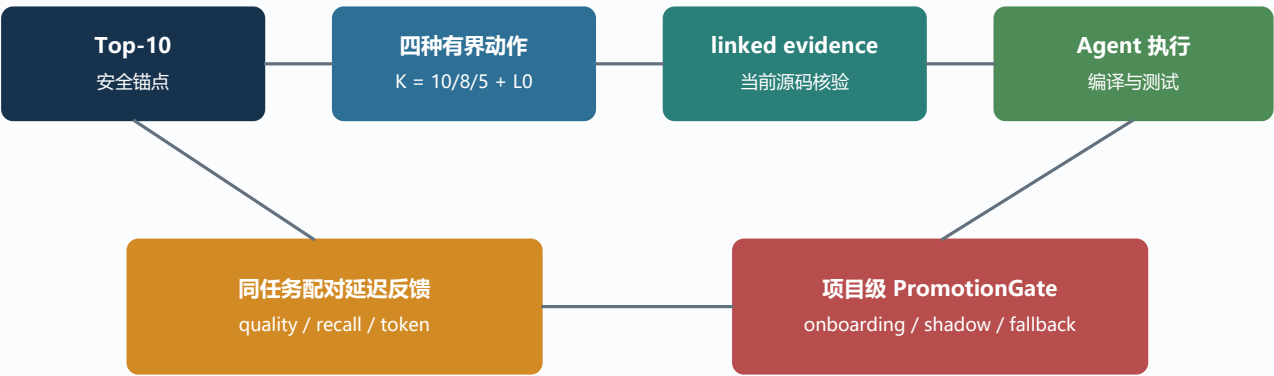
研究问题

在不改写底层 FTS5、向量检索与 RRF 的前提下，能否利用任务结束后的可执行反馈，自适应选择记忆组织动作，并在证据不足或风险出现时可靠回退？

职责边界

研究问题	研究范围
调整 Top-K 与轻量 L0 补证	重写 MemoryCore 的 FTS5、embedding 或 RRF
按 team / agent / project 隔离学习状态	使用 evaluator-only gold patch 或 hidden tests 生成补丁
用延迟配对反馈决定候选是否可晋升	把 token 节省直接等同于任务成功
用 linked 线索引导模型回到当前源码核验	把未绑定版本的函数摘要直接当作事实答案

方案闭环



warmup、状态损坏、L0 失败或硬回归时回退 Top-10

2. 最终方案

最终工程候选由四部分组成：Top-10
安全锚点、有界动作选择、同任务配对延迟反馈、按项目作用域的晋升与回退。学习器可使用经验均值或不新增 LLM 调用的 LinUCB。

组件	策略	策略约束
AdaptiveRecallPolicy	在 top10、top8、top5、top5-l0 中决策并持久化作用域状态	warmup 与 shadow 强制 Top-10；状态损坏或过期回退
CodingFeedback	将 pass@1、patch、compile、tests、regression 与 token 归一化	必须与同任务 Top-10 成对记录；质量和成本分开
PromotionGate	检查样本量、编程反馈覆盖率、质量/成本下界与硬失败	未达标不晋升；hard regression 立即降级
AutoRecall 接入	只覆盖 maxResults，top5-l0 复用已有 L0 FTS5	默认关闭；L0 查询失败时保留 L1
SWE 实验求解器	linked 定位、asserted reproduction、编辑、窄回归与回滚	与生产路径隔离，不替换原生 prompt

函数信息如何进入记忆

函数用途、参数和相关符号既可由 LLM 抽取，也可由 AST 或索引器确定性生成。当前方案没有把自由文本函数卡片直接作为答案，而是把符号、定义位置和源码路径用作 linked retrieval 线索。这样模型可以更快定位，同时仍需在当前提交源码中核验签名和行为，降低陈旧摘要造成的误导。

后续可实现结构化函数卡片，但应先加入 commit/version 绑定、AST 签名校验、引用位置和陈旧检测，再做独立消融。

3. 方案演化与筛选

实验	问题	结论/处理
固定 Top-5 / Top-8	节省 token，但不同 query 的质量变化不一致	保留为有限动作，不全局替换
L1 + L0 邻居	可补原始上下文，成本仍可控	形成 top5-l0；失败不影响 L1
KNN / 分桶	历史回放为正，小样本和分布漂移下不稳定	KNN24 保留为研究主线，不作默认
LinUCB	无需额外 selector 调用，可表达 query 差异	与 mean 并列实现，受门限保护
函数卡片	可缩短定位，但脱离当前源码可能陈旧	改为 linked 线索，保留版本化卡片方向
即时记忆	容易固化偶然错误或错误修复	改为编译/测试后的延迟反馈
Reproduction + repair	多消耗 33,626 token，5 题 resolved 仍为 1	保留验证纪律，不将更多轮次视为收益
置信下界 KNN	开发集仍选 KNN24；冻结验证 recall 下降	不晋升，停止在已观察数据上继续调参
MLP / 离线 Q	30 题样本中 MLP 过拟合，Q 的 F1 点增益伴随 recall 损失	小样本下不增加模型复杂度

筛选标准不是单次最高分，而是同时满足可归因、可回退、质量下界、证据召回下界、成本预算和实现复杂度约束。

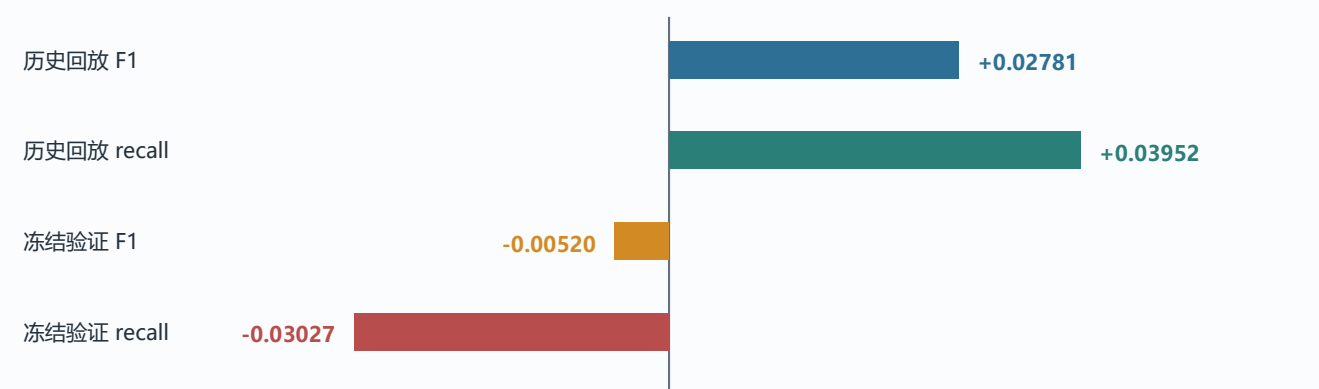
4. 长对话实验结果

历史回放覆盖 5 个 LoCoMo 对话，每个对话前 24 条用于 onboarding，之后共 923 条 query。结果相对同输入 Top-10 为正，但这些对话在方法开发期间已被观察，因此只能作为机制证据，不能称为新独立 holdout。

历史回放指标	Baseline	相对 Top-10	备注
Token F1	0.21458	+0.02781	query 与 dialog-cluster bootstrap 区间均不跨 0
Evidence recall	0.36380	+0.03952	query 与 dialog-cluster bootstrap 区间均不跨 0
平均输入 token	209.50	-63.57	节省率 23.28%

冻结切分

最终一轮固定 locomo-8/9/0 为开发集、locomo-1/7 为验证集，不新增 LLM 调用。开发集选择置信系数 0，即原 KNN24；296 条验证 query 上三个置信版本都未同时守住 F1 与 recall。



0 = Top-10; 右侧为提升，左侧为下降

置信系数	F1 变化	Recall 变化	Token 变化	备注
0 (冻结选择)	-0.00520	-0.03027	-50.59	否
0.5	+0.00429	-0.02293	-43.23	否
1.0	-0.00481	-0.02349	-33.30	否

5. 跨数据集与编程实验

LongMemEval

在 30 条缓存答案上，ridge-gated-0.01 仅对 4 条 query 使用 Top-5，F1 和 recall 差值均为 0，每题节省 22.9 token，token bootstrap 95% CI 为 [-46.37,-3.53]。样本小且存在跨数据集特征漂移，因此不把该参数迁移到 LoCoMo。

SWE 编程场景

问题	Public validation	Official resolved	API token	反馈
5 题 public reproduction	5 / 5 有效	1 / 5	114,384	基础反馈链路
reproduction + repair	5 / 5 有效	1 / 5	148,010	增加成本，未增加解决数
Matplotlib 13989	通过公开复现与窄回归	未单列	-	linked 定位恢复有效候选补丁
Astropy 14096 adaptive	记忆延迟注入成功	无有效补丁	-	触发机制有效，任务收益未证实

编程反馈与门控验证

任务	输入状态	任务
任务 1-23	样本不足	onboarding / Top-10
任务 24	质量、成本与编程覆盖率通过	promoted
随后硬失败	regression + stale evidence	立即 demoted / Top-10

Promotion smoke 证明状态机和硬回退机制，不是效果数据。当前真实编程反馈仅 2 行，低于 24 行晋升要求。

6. 证据、结论与局限

证据来源	可以支持	不能支持
单元测试与 smoke	动作、持久化、反馈、晋升和回退按设计工作	真实任务成功率提升
923 条历史回放	已观察对话上存在质量/成本正收益	新独立数据上的泛化
296 条冻结验证	候选没有通过双质量门，拒绝晋升是正确行为	继续调参即可稳定转正
30 条 LongMemEval	低风险节省 token 的局部候选	跨数据集参数迁移
5 题 SWE + 个案	linked 定位、复现和回退具有过程价值	记忆稳定提高 pass@1

最终判断

目前最合理的编程场景优化方案，是保留 Top-10 默认路径，将 KNN24 / LinUCB 作为 shadow 候选，用 linked evidence 降低定位成本，用 compile、tests、regression 和 pass@1 做延迟配对反馈，并只在跨项目证据通过双质量门后晋升。

主要局限

LoCoMo 对话已参与方法开发；LongMemEval 只有 30 条；SWE 任务数量少且模型能力构成明显上限；真实编程反馈覆盖不足；PromotionGate 目前位于实验适配/回放层，尚未默认串入 AutoRecall 的线上决策路径。

因此本报告将贡献表述为“可控的记忆优化与验证闭环”，而不是“已证明提高编程成功率”。

7. 可复现性与提交内容

精简仓库从上游 v2.0.0-beta.1 重建，仅保留题目三 C 的实现、结果、复现脚本与报告。离线冻结回放不调用 LLM，紧凑动作表不包含对话正文、答案文本、模型请求或凭据。

项目	结果
Topic 3C TypeScript	3 个测试文件，17 / 17 通过
Python SWE solver	14 / 14 通过，语法编译通过
Promotion smoke	第 24 条晋升，hard regression 后降级
冻结回放	选择系数 0；F1 -0.00519565，recall -0.03026730，token -50.5912/题
MemoryCore build	通过
ZIP 干净复现	重新安装依赖后，测试、smoke、回放和构建全部通过

最小复现命令

项目	命令
MemoryCore	npm install
MemoryCore	npm run test:topic3c
MemoryCore	npm run smoke:topic3c
MemoryCore	npm run eval:topic3c
MemoryCore	npm run build
MemoryCore	python -m pytest -q scripts/topic3-c/test_swe_agent_v2.py

安全边界

提交不包含 API key、模型请求缓存、Docker 镜像、完整 benchmark bundle、gold patch、hidden tests、FAIL_TO_PASS、PASS_TO_PASS 或 evaluator-only 数据。

8. 后续实验优先级

优先级	实验	通过标准
P0	冻结模型、预算与基线，在更多独立仓库任务比较 no-memory、Top-10、候选策略	跨项目重复出现 resolved 或公开验证收益，且回归不增加
P0	积累真实编程配对反馈，保持候选处于 shadow	达到 24 条覆盖门槛后再评估晋升
P1	按 query 类型学习 K 与 L0 配额，不新增 selector LLM	F1 与 evidence recall 下界不降，token 满足预算
P1	版本化函数卡片：AST 签名、commit 绑定、引用位置、陈旧检测	在独立消融中减少定位步数，不降低 patch validity
P2	真正按需的增量 L2/L3	只在基线证据不足时读取，并证明额外成本有回报

提交主线：我们没有用负结果否定记忆系统，而是用负结果校准启用条件。当前价值在于让记忆何时被调用、如何被验证、何时回退都成为可学习且可审计的过程。

参考材料

1. TencentDB Agent Memory v2.0.0-beta.1。
2. ReProAgent: 分阶段的定位、根因、测试规划与测试生成。
3. Self-Evolving Coding Agents: 可执行反馈、轨迹记忆、可逆性及成本/泛化门禁。
4. 仓库内 IMPLEMENTATION.md、EXPERIMENT_HISTORY.md、RESULTS.md 与 CLEANROOM.md。

报告数据冻结日期：2026-09-11。